Following the demonstration we were able to increase the encryption speed further from ~10MB/s to ~16MB/s. This design is located in the repository under a separate branch called fixed_speedup which contains the code, the binary files for the board and also the synthesis reports for the version. In the demonstrated design the system was not fully unrolled, and hence could not hit the initiation interval target of 1. This was because of a high LUT count associated with the implementation of the arrays used as lookup tables to implement the sub bytes and mix columns functions, taking ~20k and ~40k LUTs respectively for each function instance. The calls to these lookups for the demonstration utilised const arrays of the values, however, switching to a switch statement setup which assigned individual bits based on the input values which would output individual bits of the output in a logic layout which allowed the synthesis tool to greatly reduce the number of LUTs required for these stages down to ~1800 for sub bytes ~4000 for mix columns. This version did have some trouble building initially, requiring function inlining to be explicitly turned off to allow the functions to be converted with less context. This reduced the necessary resources to the point where unrolling became possible and hence the II target could be hit from the actual encryption perspective. This was expected to increase the throughput however, due to the fact that it was only a slight 1.5x it appears that the design hit another bottleneck limiting the speed increase.

From this point the bottleneck now appears to be the IO with the timing summary showing significant blocks of read and write waiting. This means that the next step to increasing performance would be increasing the amount of data read/written back in a block. Memcpy does increase the capacity for transfers and allows the synthesis tools to better utilise the AXI bus removing II errors due to arrays not being available in the same clock cycle, however, the transfers of a single block are not utilising the AXI burst mode to its full extent to cut down on the waiting time and hence buffering inputs and outputs to increase the number of blocks transferred in one burst transfer and hence reduce the number of readreq, writeresp periods which are the significant blocks in the schedule viewer.